



Autograder Assignment: Image Classification Using PyTorch

Estimated Time Needed: 60 Minutes

Overview

In this hands-on lab, you will work on image classification using PyTorch and the Intel image dataset. The assignment covers downloading and exploring the dataset, building a fully connected neural network, and implementing a convolutional neural network (CNN) for classification. You will also apply regularization techniques to improve model robustness and evaluate performance.

Learning Objectives

By the end of this assignment, you will be able to:

- Download and preprocess image data using OpenCV.
- Convert image datasets into NumPy arrays and scale them.
- Build and train a fully connected neural network using PyTorch.
- Develop a convolutional neural network (CNN) for image classification.
- Implement regularization techniques to improve model performance and robustness.

Dataset Information

The Intel image dataset consists of various categories like 'buildings,' 'forest,' 'glacier,' 'mountain,' 'sea,' and 'street.' The dataset is used for training and testing machine learning models on image classification tasks.

Problem 1: Download the dataset, unzip and explore the file folders. Load the image datasetwith training and testing grouped. (Note, cv2 reads and saves in BGR channel order) Display a few images. How many color channels are there?

```
In [1]: import os
import cv2
import numpy as np
import matplotlib.pyplot as plt

In [2]: # Unzip the images
!unzip -o seg_test.zip
Archive:  seg_test.zip
  creating:  seg_test/buildings/
  inflating: seg_test/buildings/20057.jpg
  inflating: seg_test/buildings/20060.jpg
  inflating: seg_test/buildings/20061.jpg
  inflating: seg_test/buildings/20064.jpg
  inflating: seg_test/buildings/20073.jpg
  inflating: seg_test/buildings/20074.jpg
  inflating: seg_test/buildings/20078.jpg
  inflating: seg_test/buildings/20083.jpg
  inflating: seg_test/buildings/20094.jpg
  inflating: seg_test/buildings/20096.jpg
  inflating: seg_test/buildings/20113.jpg
  inflating: seg_test/buildings/20131.jpg
  inflating: seg_test/buildings/20140.jpg
  inflating: seg_test/buildings/20177.jpg
  inflating: seg_test/buildings/20186.jpg
  inflating: seg_test/buildings/20206.jpg
  inflating: seg_test/buildings/20207.jpg

In [16]: # Paths and Labels
_labels = ['buildings', 'forest', 'glacier', 'mountain', 'sea', 'street'] # Categories
IMGSIZE = (128, 128) # Resize all images to 128x128 pixels
CNAMES = ['buildings', 'forest', 'glacier', 'mountain', 'sea', 'street'] # Duplicate of _labels for Label indexing
X_tr, y_tr = [], [] # Empty Lists to store images and Labels
def saveimages_inlist():
    """
    Function to load images from subdirectories corresponding to different categories,
    resize them to a uniform size (128x128), and return the images along with their labels.

    Returns:
    -----
    X_tr : list
        A list containing all the resized images as numpy arrays.
    y_tr : list
        A list containing the corresponding label indices of the images.

    Steps:
    1. The function defines a list of labels (_labels) which correspond to the directories
       where images are stored. Each subdirectory is named after a category (e.g., 'buildings', 'forest').
    2. The images are resized to 128x128 pixels (IMGSIZE).
    3. An empty list, X_tr, is used to store the resized images, and y_tr stores the label indices.
    4. For each label in _labels, the function constructs the path to the images of that label.
       It reads all '.jpg' files, resizes them, and appends them to the X_tr list.
       The corresponding label's index is appended to y_tr.
    5. Finally, the function returns the list of images (X_tr) and their labels (y_tr).

    Example:
    -----
    If the 'seg_train' directory contains folders 'buildings', 'forest', etc.,
    each with several images, the function will load all the images, resize them,
    and return two lists: one with the resized images and another with the corresponding labels.
    """

    X_tr, y_tr = [], []
    # Load images
    for label in _labels:
        path = os.path.join('seg_test', label) # Construct the path for each category
        # Read all .jpg images from the directory

        # Read and resize the image and append that to the List X_tr. Use cv2.resize.

        # # Append the index of the Label to y_tr
        # your code here
        #raise NotImplementedError
        img_files = [f for f in os.listdir(path) if f.lower().endswith(".jpg")]
        X_tr += [cv2.resize(cv2.imread(os.path.join(path, f)), IMGSIZE) for f in img_files]
        y_tr += [_labels.index(label)]*len(img_files)

    return X_tr, y_tr # Return the images and their corresponding Label indices

In [18]: # Note: This is a read-only cell and cannot be edited.
X_tr,y_tr=saveimages_inlist()
# Check the number of color channels
print ("Number of channels=",X_tr[0].shape[2])
# Display a few images
plt.figure(figsize=(10, 10))
for i in range(9):
    plt.subplot(3, 3, i + 1)
    plt.imshow(cv2.cvtColor(X_tr[i], cv2.COLOR_BGR2RGB)) # Convert BGR to RGB
    plt.title(CNAMES[y_tr[i]])
    plt.axis('off')
plt.show()

Number of channels= 3
```



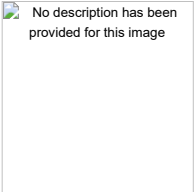
Grades

Close X

Passed • Grade Received: 100%

Submissions

August 18, 2025 1:29 PM IST (Highest Grade) ▾



## Autograder Assignment: Image Classification Using PyTorch¶

Estimated Time Needed: 60 Minutes

### Overview¶

In this hands-on lab, you will work on image classification using PyTorch and the Intel image dataset. The assignment covers downloading and exploring the dataset, building a fully connected neural network, and implementing a convolutional neural network (CNN) for classification. You will also apply regularization techniques to improve model robustness and evaluate performance.

### Learning Objectives¶

By the end of this assignment, you will be able to:

- Download and preprocess image data using OpenCV.
- Convert image datasets into NumPy arrays and scale them.
- Build and train a fully connected neural network using PyTorch.
- Develop a convolutional neural network (CNN) for image classification.
- Implement regularization techniques to improve model performance and robustness.

### Dataset Information¶

The Intel image dataset consists of various categories like 'buildings,' 'forest,' 'glacier,' 'mountain,' 'sea,' and 'street.' The dataset is used for training and testing machine learning models on image classification tasks.

Problem 1: Download the dataset, unzip and explore the file folders. Load the image datasetwith training and testing grouped. (Note, cv2 reads and saves in BGR channel order) Display a few images. How many color channels are there?¶

In [1]:

```
import os
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

In [2]:

```
# Unzip the images
!unzip -o seg_test.zip
```

In [3]:

Student's answer (Top)

```
# Paths and Labels
_labels = ['buildings', 'forest', 'glacier', 'mountain', 'sea', 'street'] # Categories
IMGSIZE = (128, 128) # Resize all images to 128x128 pixels
CNAMES = ['buildings', 'forest', 'glacier', 'mountain', 'sea', 'street'] # Duplicate
X_tr, y_tr = [], [] # Empty Lists to store images and Labels
def saveimages_inlist():
    """
    Function to Load images from subdirectories corresponding to different categories,
    resize them to a uniform size (128x128), and return the images along with their la

    Returns:
    -----
    X_tr : List
        A list containing all the resized images as numpy arrays.
    y_tr : List
        A list containing the corresponding Label indices of the images.

    Steps:
    1. The function defines a List of Labels (_Labels) which correspond to the directo
       where images are stored. Each subdirectory is named after a category (e.g., 'bu
    2. The images are resized to 128x128 pixels (IMGSIZE).
    3. An empty list, X_tr, is used to store the resized images, and y_tr stores the L
    4. For each label in _labels, the function constructs the path to the images of th
       It reads all '.jpg' files, resizes them, and appends them to the X_tr list.
       The corresponding Label's index is appended to y_tr.
    5. Finally, the function returns the List of images (X_tr) and their Labels (y_tr)

    Example:
    -----
    If the 'seg_train' directory contains folders 'buildings', 'forest', etc.,
    each with several images, the function will Load all the images, resize them,
    and return two lists: one with the resized images and another with the correspondi
    """

    X_tr, y_tr = [], []
    # Load images
    for label in _labels:
        path = os.path.join('seg_test', label) # Construct the path for each category
        # Read all .jpg images from the directory

        # Read and resize the image and append that to the List X_tr. Use cv2.resize.

        # # Append the index of the Label to y_tr
        # your code here
        #raise NotImplementedError
        img_files = [f for f in os.listdir(path) if f.lower().endswith(".jpg")]
        X_tr += [cv2.resize(cv2.imread(os.path.join(path, f)), IMGSIZE) for f in img_f
        y_tr += [_labels.index(label)]*len(img_files)

    return X_tr, y_tr # Return the images and their corresponding Label indices
```

In [4]:

Grade cell: cell-a084b9aa65337625 Score: 25.0 / 25.0 (Top)

```
# Note: This is a read-only cell and cannot be edited.
X_tr,y_tr=saveimages_inlist()
Hidden Tests Redacted
# Check the number of color channels
print ("Number of channels=",X_tr[0].shape[2])
# Display a few images
plt.figure(figsize=(10, 10))
for i in range(9):
    plt.subplot(3, 3, i + 1)
    plt.imshow(cv2.cvtColor(X_tr[i], cv2.COLOR_BGR2RGB)) # Convert BGR to RGB
    plt.title(CNAMES[y_tr[i]])
    plt.axis('off')
plt.show()

Congratulations! All test cases in this cell passed.
```

Problem 2: Convert the image dataset into a numpy array and scale the images to values between [0, 1].¶

In [5]:

Student's answer (Top)

```
def convert_toNumpyarray(X_tr, y_tr):
    """
    Function to convert the Lists of images and Labels into numpy arrays and normalize

    Parameters:
    -----
    X_tr : List
        A list containing image data, where each image is represented as a 3D array (h
    y_tr : List
        A list containing the corresponding Label indices for each image.

    Returns:
    -----
    X_tr : numpy.ndarray
        A numpy array containing all the image data, normalized to a range of [0, 1].
    y_tr : numpy.ndarray
        A numpy array containing the Label indices.

    Steps:
    -----
    1. Converts the List `X_tr`, which contains the images, into a numpy array with dt
    2. Normalizes the image data by dividing the pixel values by 255, scaling the pixe
    3. Converts the List `y_tr`, which contains the Label indices, into a numpy array.

    Example:
    -----
    If `X_tr` is a List of resized images and `y_tr` contains the corresponding Labels
    this function will convert both Lists to numpy arrays and normalize the pixel valu
    """

    # Convert X_tr (images) to a numpy array and scale pixel values to the range [0, 1
    # Convert y_tr (Labels) to a numpy array
    # your code here
    #raise NotImplementedError
    X_tr, y_tr = np.array(X_tr), np.array(y_tr)
    X_tr = X_tr / X_tr.max()

    return X_tr, y_tr # Return the normalized images and Label arrays
```

In [6]:

Grade cell: cell-e1ce61350f609899 Score: 25.0 / 25.0 (Top)

# Note: This is a read-only cell and cannot be edited.  
X\_tr,y\_tr=convert\_toNumpyarray(X\_tr, y\_tr)

Hidden Tests Redacted

print("X\_tr.shape=",X\_tr.shape)  
print("y\_tr.shape=",y\_tr.shape[0])

Congratulations! All test cases in this cell passed.

Problem 3:Create a fully connected neural network and report its performance on the dataset.

- Define a simple neural network with 3 fully connected layers.
- Train it for 10 epochs and report the loss and accuracy.

In [7]:

```
!pip install torch
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

In [8]:

Student's answer

(Top)

```
class SimpleNN(nn.Module):
    """
    A simple feedforward neural network for classification tasks.

    This model consists of three fully connected layers with ReLU activations.
    It is designed to process images of a specific size (e.g., 128x128 pixels with 3 c

    Attributes:
        fc1 (nn.Linear): The first fully connected layer, which takes flattened image
        fc2 (nn.Linear): The second fully connected layer, which takes the 512 feature
        fc3 (nn.Linear): The output layer, which takes the 256 features from the secon
    """

    def __init__(self):
        """
        Initializes the SimpleNN class.

        The layers are defined and initialized:
        - Three fully connected layers for the feedforward network.
        - The input size is adjusted based on the expected flattened image size.
        """
        super(SimpleNN, self).__init__() # Initialize the parent class

        # Define the layers
        self.fc1 = nn.Linear(3 * 128 * 128, 512) # First fully connected layer (input
        self.fc2 = nn.Linear(512, 256) # Second fully connected layer (input
        self.fc3 = nn.Linear(256, 10) # Output layer (input: 256 neurons,

    def forward(self, x):
        """
        Defines the forward pass of the network.

        Args:
            x (torch.Tensor): Input tensor of shape (batch_size, channels, height, wid
            where channels = 3 for RGB images, and height and width

        Returns:
            torch.Tensor: Output predictions of shape (batch_size, number_of_classes),
            containing the logits for each class.
        """
        # Flatten the input tensor to prepare for the fully connected layers
        x = x.reshape(x.size(0), -1) # Reshape to (batch_size, -1) to flatten the ten

        # Pass through the first fully connected layer and apply ReLU activation
        x = torch.relu(self.fc1(x))

        # Pass through the second fully connected layer and apply ReLU activation
        x = torch.relu(self.fc2(x))

        # Pass through the output layer to get predictions
        x = self.fc3(x) # Output layer (final predictions)

        return x # Return the output predictions

def test_fully_connected_network(X_tr, y_tr):
    """
    Trains a fully connected neural network on the given training data and evaluates i

    Parameters:
        X_tr (numpy.ndarray): Training data features (input).
        y_tr (numpy.ndarray): Training data labels (output).

    Returns:
        float: The validation accuracy after training.
    """

    # Split data into training and validation sets (80-20 split)
    X_train, X_val, y_train, y_val = train_test_split(X_tr, y_tr, test_size=0.2, rando

    # Convert data to PyTorch tensors and move to the appropriate device (CPU or GPU)
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu") # Use GPU if
    X_train_tensor = torch.tensor(X_train, dtype=torch.float32).to(device) # Training
    y_train_tensor = torch.tensor(y_train, dtype=torch.long).to(device) # Trainin
    X_val_tensor = torch.tensor(X_val, dtype=torch.float32).to(device) # Validati
    y_val_tensor = torch.tensor(y_val, dtype=torch.long).to(device) # Validat

    # Initialize the simple neural network
    input_size = X_train_tensor.shape[1] # Determine the input size from the training
    num_classes = len(torch.unique(y_train_tensor)) # Count the number of unique clas
    model = SimpleNN().to(device) # Create the model and move it to the device
    criterion = nn.CrossEntropyLoss() # Loss function for multi-class classification
    optimizer = optim.Adam(model.parameters(), lr=0.001) # Adam optimizer with Learnin

    # Training Loop
    num_epochs = 10 # Set the desired number of epochs for training
    batch_size = 32 # Define a batch size for training

    for epoch in range(num_epochs):
        model.train() # Set the model to training mode

        # Shuffle training data for each epoch to improve training efficiency
        indices = torch.randperm(X_train_tensor.size(0)) # Generate random indices for
        X_train_tensor = X_train_tensor[indices] # Shuffle the training features
        y_train_tensor = y_train_tensor[indices] # Shuffle the training labels

        # Mini-batch training
        for i in range(0, X_train_tensor.size(0), batch_size):
            batch_X = X_train_tensor[i:i + batch_size] # Get a mini-batch of training
            batch_y = y_train_tensor[i:i + batch_size] # Corresponding labels for the

            # Zero the parameter gradients for the optimizer
            # Forward pass: Compute predicted outputs by passing inputs to the model
            # Compute the loss
            # Backward pass: Compute the gradient of the loss with respect to model pa
            # Update the model parameters based on the gradients

            # your code here
            #raise NotImplementedError
            optimizer.zero_grad()
            outputs = model(batch_X)
            loss = criterion(outputs, batch_y)
            loss.backward()
            optimizer.step()

            # Clear unused variables to manage memory
            del batch_X, batch_y, outputs, loss # Delete tensors to free up memory
            torch.cuda.empty_cache() # Free up GPU memory if using CUDA

        # Validation step
        model.eval() # Set the model to evaluation mode
        with torch.no_grad(): # Disable gradient computation during validation
            val_outputs = model(X_val_tensor) # Get the model output for the validati
            val_preds = torch.argmax(val_outputs, dim=1) # Predict the class with the
            val_accuracy = accuracy_score(y_val, val_preds.cpu().numpy()) # Calculate

        # Print the validation accuracy after each epoch
        print(f'Epoch [{epoch + 1}/{num_epochs}], Validation Accuracy: {val_accuracy:.

    # Return the final validation accuracy
    return val_accuracy
```

In [9]:

Grade cell: cell-fb76a1f54309dfe8

Score: 25.0 / 25.0 (Top)

# Note: This is a read-only cell and cannot be edited.  
val\_accuracy = test\_fully\_connected\_network(X\_tr, y\_tr)

Hidden Tests Redacted

print("Validation accuracy=",val\_accuracy)

Congratulations! All test cases in this cell passed.

Problem 4: Create a convolutional neural network with dropout and regularization, and report its performance.

- Define a CNN with 2 convolutional layers and 2 fully connected layers.
- Add dropout after the fully connected layers to prevent overfitting.
- Train the model and report the validation accuracy.

In [10]:

Student's answer

(Top)

```
class RegularizedCNN(nn.Module):
    """
    A Convolutional Neural Network (CNN) with regularization to classify images.

    This model consists of two convolutional layers followed by max pooling,
    a couple of fully connected layers, and dropout for regularization.

    Attributes:
    - conv1: First convolutional layer.
    - conv2: Second convolutional layer.
    - pool: Max pooling layer to downsample feature maps.
    - fc1: First fully connected layer.
    - fc2: Output layer with neurons equal to the number of classes.
    - dropout: Dropout layer for regularization.
    """

    def __init__(self):
        """
        Initializes the RegularizedCNN class.

        The layers are defined and initialized:
        - Two convolutional layers with ReLU activations and padding.
        - A max pooling layer to reduce dimensionality.
        - Two fully connected layers for classification.
        - A dropout layer to help prevent overfitting.
        """
        super(RegularizedCNN, self).__init__() # Initialize the parent class

        # Define the layers
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1) # First convolutional
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1) # Second convolution
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2) # Max pooling layer (kernel
        self.fc1 = nn.Linear(64 * 32 * 32, 128) # First fully connected layer (input:
        self.fc2 = nn.Linear(128, len(CNAMES)) # Output layer (input: 128 neurons, ou
        self.dropout = nn.Dropout(0.5) # Dropout layer with 50% drop probability

    def forward(self, x):
        """
        Defines the forward pass of the network.

        Args:
        - x (torch.Tensor): Input tensor of shape (batch_size, channels, height, width)

        Returns:
        torch.Tensor: Output predictions of shape (batch_size, number_of_classes).
        """
        # Pass through the first convolutional layer and apply ReLU and max pooling
        x = self.pool(torch.relu(self.conv1(x)))

        # Pass through the second convolutional layer and apply ReLU and max pooling
        x = self.pool(torch.relu(self.conv2(x)))

        # Flatten the tensor to prepare for fully connected layers
        x = x.reshape(x.size(0), -1) # Reshape to (batch_size, -1)

        # Pass through the first fully connected layer, apply ReLU, and dropout
        x = self.dropout(torch.relu(self.fc1(x)))

        # Pass through the output layer to get predictions
        return self.fc2(x) # Output layer (final predictions)

def test_convolution_network(X_tr, y_tr):
    """
    Trains a convolutional neural network on the given training data and evaluates it

    Parameters:
    X_tr (numpy.ndarray): Training data features, expected to have shape (num_samples,
    y_tr (numpy.ndarray): Training data labels, expected to be a 1D array of class ind

    Returns:
    float: The validation accuracy after training.
    """

    # Split data into training and validation sets (80-20 split)
    X_train, X_val, y_train, y_val = train_test_split(X_tr, y_tr, test_size=0.2, random

    # Convert data to PyTorch tensors
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu") # Use GPU if
    X_train_tensor = torch.tensor(X_train, dtype=torch.float32).permute(0, 3, 1, 2).to
    y_train_tensor = torch.tensor(y_train, dtype=torch.long).to(device) # Convert lab
    X_val_tensor = torch.tensor(X_val, dtype=torch.float32).permute(0, 3, 1, 2).to(dev
    y_val_tensor = torch.tensor(y_val, dtype=torch.long).to(device) # Validation labe

    # Initialize the convolutional neural network model and optimizer
    model = RegularizedCNN().to(device) # Instantiate the model and move it to the sp
    optimizer = optim.Adam(model.parameters(), lr=0.001) # Adam optimizer with a lear
    criterion = nn.CrossEntropyLoss() # Define the loss function for multi-class clas
    batch_size = 32 # Set the batch size for training
    num_epochs = 10 # Set the number of training epochs

    # Training loop for the specified number of epochs
    for epoch in range(num_epochs):
        model.train() # Set the model to training mode

        # Shuffle training data for each epoch to improve training efficiency
        indices = torch.randperm(X_train_tensor.size(0)) # Generate a random permutat
        X_train_tensor = X_train_tensor[indices] # Shuffle training data
        y_train_tensor = y_train_tensor[indices] # Shuffle training labels

        # Mini-batch training
        for i in range(0, X_train_tensor.size(0), batch_size):

            # Get a mini-batch of training data and assign it to a variable batch_X
            # Get corresponding labels for the mini-batch and assign it to a variable
            # Zero the parameter gradients for the optimizer
            # Forward pass: Compute predicted outputs by passing inputs to the model
            # Compute the loss
            # Backward pass: Compute gradient of the loss with respect to model paramet
            # Backpropagate the loss
            # Update the model parameters based on the gradients

            # your code here
            raise NotImplementedError

            batch_X = X_train_tensor[i:i + batch_size] # Get a mini-batch of training
            batch_y = y_train_tensor[i:i + batch_size] # Corresponding labels for the

            optimizer.zero_grad()
            outputs = model(batch_X)
            loss = criterion(outputs, batch_y)
            loss.backward()
            optimizer.step()

            # Clear unused variables to manage memory
            del batch_X, batch_y, outputs, loss # Delete tensors to free up memory
            torch.cuda.empty_cache() # Free up GPU memory if using CUDA

        # Validation step
        model.eval() # Set the model to evaluation mode
        with torch.no_grad(): # Disable gradient computation during validation
            val_outputs = model(X_val_tensor) # Get the model output for the validati
            val_preds = torch.argmax(val_outputs, dim=1) # Predict the class with the
            val_accuracy = accuracy_score(y_val, val_preds.cpu().numpy()) # Calculate

        # Print the validation accuracy after each epoch
        print(f'Epoch [{epoch + 1}/{num_epochs}], Validation Accuracy: {val_accuracy:.

    # Return the final validation accuracy
    return val_accuracy
```

In [11]:

Grade cell: cell-36984ae2616a30bf

Score: 25.0 / 25.0 (Top)

```
# Note: This is a read-only cell and cannot be edited.
val_accuracy = test_convolution_network(X_tr, y_tr)

Hidden Tests Redacted

print("Validation accuracy=", val_accuracy)
```

Congratulations! All test cases in this cell passed.

Congratulations! for completing this Autograded Assignment.